

OSI Model & Modular Monolith

OSI Model Explained Through a NestJS Modular Monolith

Imagine this flow:

A frontend/mobile client sends POST /orders to your NestJS backend → your app authenticates the user → validates DTO → calls service logic → writes to PostgreSQL → returns JSON.

That single request travels through **all 7 OSI layers**.

First: What is a modular monolith in NestJS?

In a **NestJS modular monolith**, your application is deployed as **one application/process (or a small set of identical app instances)**, but internally organized into **feature modules** like:

- AuthModule
- UserModule
- OrderModule
- PaymentModule
- NotificationModule

So the architecture is **logically modular**, but **physically one deployable unit**.

That matters because the OSI model describes **how data moves through the network stack**, while modular monolith describes **how your application code is structured once the request reaches the app**.

The 7 OSI Layers Mapped to NestJS

7) Application Layer — “What business-level protocol is the client using?”

In general

This is the layer where protocols like:

- HTTP / HTTPS
- WebSocket
- gRPC

- **SMTP**, etc.

are used by applications.

In your NestJS backend

This is the layer you interact with the most as a backend engineer.

Examples:

- GET /users/:id
- POST /auth/login
- REST API responses in JSON
- GraphQL resolvers
- WebSocket gateway events

NestJS concepts at this layer

These are all **Application-layer concerns**:

- **Controllers**
- **Routes**
- **Guards**
- **Interceptors**
- **Pipes**
- **Exception filters**
- **DTO validation**
- **Serialization**
- **Auth headers / cookies**
- **Response bodies (JSON)**

Example

```
@Controller('orders')
export class OrdersController {
  @Post()
  createOrder(@Body() dto: CreateOrderDto) {
    return this.ordersService.create(dto);
  }
}
```

```
}
```

From the OSI perspective, this is the part where:

- the client says: “I want to call POST /orders”
- your backend says: “Here is a 201 Created with JSON”

In a modular monolith

The request enters a single NestJS app, then gets routed internally to the correct module:

- OrdersModule
- AuthModule
- UsersModule

So the **Application layer** is where your **feature modules expose capabilities**.

Think of it as: “**What API is my app offering to the outside world?**”

6) Presentation Layer — “How is the data represented?”

In general

This layer deals with:

- data format translation
- encoding/decoding
- serialization/deserialization
- compression
- encryption/decryption (in some interpretations)

In your NestJS backend

This is where data is transformed into a format both sides understand.

Examples:

- JSON serialization
- converting request body to DTO class
- validation/transformation via class-transformer / class-validator
- gzip/br compression
- UTF-8 encoding
- sometimes TLS-related data interpretation is conceptually discussed here

NestJS concepts at this layer

Examples:

- DTOs
- ValidationPipe
- ClassSerializerInterceptor
- JSON conversion
- date/string formatting
- response transformation

Example

A client sends:

```
{  
  "productId": "42",  
  "quantity": "2"  
}
```

Your NestJS app may transform this into:

```
class CreateOrderDto {  
  @IsInt()  
  productId: number;  
  
  @IsInt()  
  quantity: number;  
}
```

With transformation enabled, strings may be converted into numbers before the service sees them.

In a modular monolith

Each module may define its own presentation contracts:

- AuthModule → login payloads
- OrdersModule → order DTOs
- UsersModule → public user response shape

So even though the app is one deployable unit, **data contracts are modular**.

Think of it as: “**How is the message shaped so my NestJS code can understand it?**”

5) Session Layer — “How is the conversation maintained?”

In general

This layer is about:

- starting, maintaining, and ending sessions
- connection state
- coordination between two communicating systems

In real-world web systems, this layer is often not exposed separately because many responsibilities are handled by **TCP**, **TLS**, and the application itself.

In your NestJS backend

This shows up when you manage **stateful communication** or **user continuity**.

Examples:

- user login sessions
- cookie-based authentication
- JWT refresh token flows
- WebSocket connection lifecycle
- long-lived client connections
- session store in Redis

NestJS concepts at this layer

- session middleware
- Passport session auth
- socket connection state
- refresh token tracking
- request correlation / tracing context

Example

A user logs in:

1. server verifies credentials
2. server creates session or issues JWT

3. subsequent requests reuse that established identity context

If you use:

- express-session
- Redis-backed sessions
- WebSocket gateways with connected clients

...you are touching session-like concerns.

In a modular monolith

Because all modules live in one app, session context can be shared consistently:

- AuthModule establishes identity
- OrdersModule uses that identity
- PaymentsModule checks same user context

So session state acts like a **shared conversation context across modules**.

Think of it as: **“How does the server remember or maintain continuity across requests or connections?”**

4) Transport Layer — “How does the message reliably reach the backend process?”

In general

This layer handles:

- end-to-end communication
- segmentation/reassembly
- reliability
- flow control
- ports

Main protocols:

- **TCP**
- **UDP**

In your NestJS backend

This is extremely relevant.

Your NestJS app typically listens on a TCP port:

- 3000
- 443 behind reverse proxy
- 80

When a client calls your API, the data usually arrives over:

- **TCP** for HTTP/HTTPS
- possibly **UDP** in edge cases (not typical for standard Nest REST APIs)

What this means in practice

The transport layer ensures:

- the client connects to the correct backend port
- packets are reassembled in order
- retries and acknowledgments happen (TCP)
- the application receives a clean byte stream

NestJS perspective

Your code does not usually manage TCP directly, but it depends on it completely.

Examples:

- `app.listen(3000)`
- Nginx forwarding traffic to NestJS
- load balancer sending TCP connections to app instances
- keep-alive connections
- timeouts

In a modular monolith

This is where modular monolith differs from microservices:

Modular monolith:

- most feature-to-feature communication is **in-process**
- `OrdersService` can call `PaymentsService` directly through DI
- no network call required between internal modules

Microservices:

- one service might call another over TCP/HTTP/gRPC/message bus

So in a NestJS modular monolith, **Transport layer is mostly about client ↔ app communication**, not module ↔ module communication.

Think of it as: “**Which port/process gets the request, and how is reliable delivery handled?**”

3) Network Layer — “How does the request find the server machine?”

In general

This layer is responsible for:

- logical addressing
- routing between networks
- packet forwarding

Main protocols:

- **IP**
- **ICMP**
- routing protocols like BGP (internet-scale)

In your NestJS backend

This layer matters when traffic travels from client to your deployment environment.

Examples:

- client resolves your domain to an IP
- packets are routed across the internet
- cloud VPC/network rules decide where traffic can go
- Kubernetes/node/VM networking
- private subnet vs public subnet
- firewall/security group rules

Real-world example

If your NestJS app is deployed at:

api.example.com

Then the client ultimately reaches:

- a public IP for a load balancer
- then maybe a private IP of a VM/container

- then traffic is forwarded to your NestJS process

In a modular monolith

The app may be monolithic from a code standpoint, but at Layer 3 it still depends on infrastructure:

- VM IP
- container IP
- reverse proxy IP
- database private IP
- Redis private IP

Also, your monolith may talk to:

- PostgreSQL on one private IP
- Redis on another private IP

So even a monolith is networked once deployed.

Think of it as: **“Which machine does this request need to reach, and how do routers get it there?”**

2) Data Link Layer — “How does data move within the local network?”

In general

This layer handles:

- local network delivery
- MAC addressing
- switching
- framing
- error detection on local links

Common technologies:

- **Ethernet**
- **Wi-Fi**
- VLANs

In your NestJS backend

You rarely think about this in application code, but it still exists under everything.

Examples:

- communication between your VM and the host network
- container bridge networking
- Ethernet frames inside a data center
- LAN communication between reverse proxy and app server on the same subnet
- ARP used to resolve IP to MAC on local networks

Why it matters

If your NestJS server cannot talk to PostgreSQL in the same subnet, the issue might not be in NestJS at all — it could be:

- switch/VLAN issue
- bad NIC
- MAC learning issue
- ARP issue
- local bridge/docker network issue

In a modular monolith

Because internal modules talk in-process, Layer 2 is irrelevant **inside the application boundary**.

But it still matters for:

- app ↔ DB
- app ↔ Redis
- app ↔ load balancer
- container ↔ host network

Think of it as: **“How do devices on the same local network segment pass frames to each other?”**

1) Physical Layer — “How do bits physically travel?”

In general

This is the lowest layer:

- electrical signals
- light pulses in fiber

- radio waves in Wi-Fi
- cables, NICs, switches, hardware

In your NestJS backend

You almost never interact with this directly, but your app is impossible without it.

Examples:

- data center fiber links
- Ethernet cable
- Wi-Fi from client device
- switch ports
- server NIC hardware
- cloud provider physical infrastructure

Why backend engineers should still care

If a physical network issue happens:

- packet loss increases
- requests timeout
- DB connections drop
- API latency spikes

And from your NestJS app, it may look like:

- intermittent connection resets
- socket hang up
- timeout on DB/Redis
- health checks failing

In a modular monolith

No difference architecturally here — whether monolith or microservices, every networked system depends on the physical layer.

Think of it as: “**What actual medium carries the bits?**”

A Full Request Walkthrough in OSI Terms

Let’s trace a request to a NestJS modular monolith:

Scenario

A mobile app sends:

POST /auth/login

Host: api.example.com

Content-Type: application/json

with JSON body:

```
{  
  "email": "faiyaz@example.com",  
  "password": "secret"  
}
```

Layer-by-layer view

7. Application

- The client uses **HTTP**
- NestJS route /auth/login is invoked
- AuthController receives the request

6. Presentation

- JSON is parsed
- request body becomes LoginDto
- validation occurs
- response is serialized back to JSON

5. Session

- the login may create:
 - a session cookie, or
 - JWT / refresh token pair
- future requests reuse identity context

4. Transport

- the request travels over **TCP**
- destination port is likely 443 (HTTPS) or forwarded to 3000

3. Network

- packets are routed via **IP**
- traffic reaches load balancer / VM / container hosting the monolith

2. Data Link

- frames move across local network interfaces
- switches/NICs handle local delivery

1. Physical

- bits travel over cable/fiber/radio as electrical or optical signals
-

The Most Important Insight for a NestJS Developer

A very practical way to think about it is this:

Layers 7–5: “Application-facing concerns”

These are where backend developers spend most of their time:

- API design
- DTO validation
- authentication
- session management
- WebSocket lifecycle

Layers 4–1: “Infrastructure/network-facing concerns”

These are where ops/platform/network layers dominate:

- TCP ports
 - routing
 - local networking
 - hardware/network media
-

How Modular Monolith Changes the Picture

This is the key architectural angle.

In a NestJS modular monolith:

Internal feature communication is usually:

- **function calls**
- **dependency injection**
- **shared services/events inside one codebase/process**

Example:

OrdersService -> InventoryService -> PaymentService

This often **does not cross OSI network layers** internally, because it happens in memory inside the same app runtime.

That means:

External communication

Does use OSI layers

- client → NestJS app
- NestJS app → PostgreSQL
- NestJS app → Redis
- NestJS app → external email/payment APIs

Internal module communication

Usually **does not use network transport**

- OrdersModule calling PaymentsModule via DI is not a network event
- it is an application architecture concern, not an OSI communication event

This is one of the biggest performance and simplicity benefits of a modular monolith.

A Simple Mapping Cheat Sheet

NestJS modular monolith vs OSI

Layer 7 — Application

- HTTP routes
- REST/GraphQL/WebSocket APIs
- Controllers and handlers

Layer 6 — Presentation

- JSON
- DTO transformation

- validation
- serialization
- compression

Layer 5 — Session

- login sessions
- JWT continuity
- socket connection state
- Redis session storage

Layer 4 — Transport

- TCP
- app ports
- reliable delivery
- reverse proxy forwarding

Layer 3 — Network

- IP routing
- subnets
- VPC/network policies
- load balancer to instance routing

Layer 2 — Data Link

- Ethernet/Wi-Fi
- MAC addressing
- local switch/bridge/container network

Layer 1 — Physical

- cable/fiber/radio
- NICs
- data center hardware

Interview-Friendly Summary

If someone asks you this in an interview, you can say:

In a NestJS modular monolith, the upper OSI layers are the easiest to observe directly: the Application layer maps to HTTP routes and controllers, the Presentation layer maps to JSON serialization and DTO validation, and the Session layer maps to authentication/session continuity. The lower layers support delivery of requests to the backend: TCP at the Transport layer, IP routing at the Network layer, local LAN delivery at the Data Link layer, and physical media at the Physical layer. A key distinction is that communication between internal NestJS modules in a modular monolith is usually in-process, so it does not traverse the network stack the same way inter-service communication in a microservices architecture would.

Final mental model

Use this sentence to remember it:

OSI explains how the request reaches your NestJS app; modular monolith explains how your code handles the request once it arrives.

If you want, I can also turn this into one of these next:

1. **a concise interview answer (2–3 minutes)**
2. **a diagram showing request flow through OSI + NestJS modules**
3. **a comparison: OSI model in modular monolith vs microservices**
4. **a real request lifecycle example: Client → Nginx → NestJS → PostgreSQL**